Continuous Deployment (CD) & Kubernetes

Lecture 4

greenwich.edu.vn

UNIVERSITY OF GREENWICH

Alliance with FPT Education

Lesson's Agenda

Section 1: Continuous Development

Section 2: Kubernetes Fundamentals

Section 3: Docker Compose vs Kubernetes

Section 4: Wrap Up

Section 1: Continuous Deployment

Pipeline Recap

CI (Lesson 3): Code ⇨ Test ⇨ Docker Image ⇨ Registry (Hub)

Missing Link: The code is in the registry, but the User can't see it yet.

CD (Today): Docker Registry ⇨ Production Server.

UNIVERSITY OF GREENWICH
Alliance with FPT Education

What is CD ?

Delivery & Deployment

Continuous Delivery: Artifact is ready. Deployment is one click away (Manual approval).

Continuous Deployment: Artifact is automatically pushed to production if tests pass. Zero human touch.

We will build Continuous Deployment.

Deployment Strategies

How to deploy ?

Recreate: Stop old ⇨ Start new (Downtime).

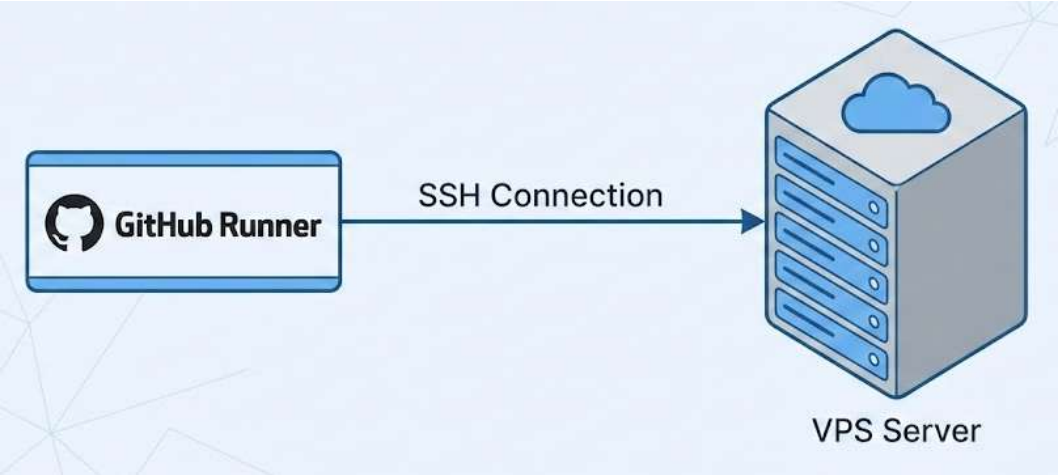
Rolling Update: Update one by one (Zero downtime).

Blue/Green: Spin up new env ⇨ Switch traffic (Safe).

The SSH Deployment Strategy

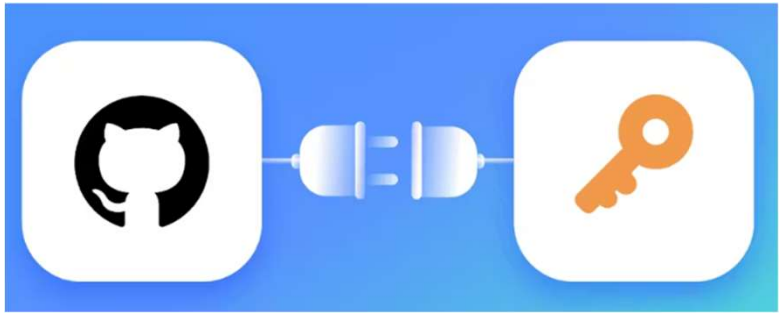
Simple CD Architecture

- Since we use Docker Compose, the easiest way is to SSH into the server and run commands remotely.
- Never store passwords or private keys in the repository; always use GitHub Secrets



SSH Keys and Security

- **SSH Keys**: Secure method for accessing remote servers.
- **GitHub Secrets** store **SSH keys** and other credentials.
- Public keys are stored on the server, private keys in GitHub Secrets.
- Ensures **secure, passwordless access** to deploy servers



GitHub Actions

Deploy job (CD)

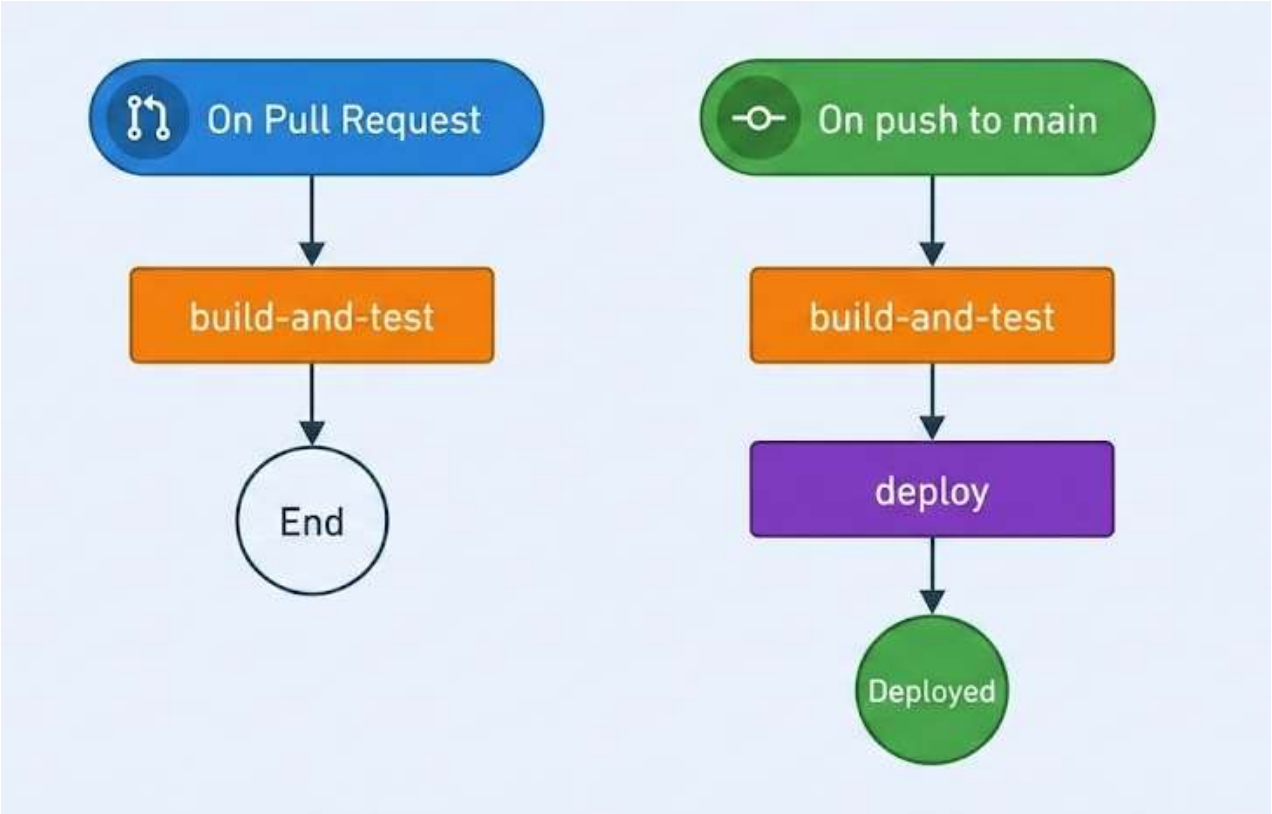
- **needs: build-and-test** enforces *"CI passes before CD"*
- **if: github.ref == 'refs/heads/main'** prevents deployments on PR branches

```yaml
deploy:
  needs: build-and-test
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest

  steps:
    - name: Deploy via SSH
      uses: appleboy/ssh-action@v1
      with:
        host: ${{ secrets.HOST }}
        username: ${{ secrets.USERNAME }}
        key: ${{ secrets.SSH_PRIVATE_KEY }}
        script: |
          cd /path/to/your-project
          git pull origin main
          docker-compose pull
          docker-compose up -d
```

End-to-end Pipeline

CI + CD workflow



Prerequisites

What you need ?

- A Cloud Server (VPS) with Public IP.
- Docker & Docker Compose installed on the server.
- SSH Key pair (Private Key for GitHub, Public Key for Server).

Security: SSH Keys

Understanding SSH Keys

- We don't want to put the server password in GitHub.
- We use SSH Keys.
- **Public Key:** Put on the Server (~/.ssh/authorized_keys). "The Lock".
- **Private Key:** Put in GitHub Secrets. "The Key".

The CD Job (YAML)

The Deploy Job

- This GitHub Actions deploy job does the following:
  1. **runs-on: ubuntu-latest** – Runs on an Ubuntu runner.
  2. **need: build-and-test** – Only runs after the build-and-test job succeeds.
  3. **uses: appleboy/ssh-action@master** – Uses the appleboy/ssh-action to SSH into the server.
  4. **with:**
     - **host** and **key** are secrets for SSH connection details.
  5. **script:**
     - **cd /app:** Navigate to the app directory.
     - **docker compose pull**: Pull the latest Docker images.
     - **docker compose up -d**: Start the containers in detached mode.
- This workflow automates deployment to a remote server using SSH and Docker.

```yaml
deploy:
  need: build # Run only after build succeeds
  runs-on: ubuntu-latest
  steps:
    - name: SSH and Deploy
      uses: appleboy/ssh-action@master
      with:
        host: ${{ secrets.SSH_HOST }}
        key: ${{ secrets.SSH_KEY }}
        script: |
          cd /app
          docker compose pull
          docker compose up -d
```

The Limitations of Compose

Docker Compose is great for single servers.

Problem:

What if the server crashes? (Downtime).

What if traffic doubles? (Cannot auto-scale).

Zero-downtime updates are hard to script manually.

Solution: Kubernetes

When do we need Kubernetes ?

Needed when running many containers (e.g., 100+) across multiple servers

Provides auto-healing, scaling, rolling updates, and stable Service endpoints

UNIVERSITY OF GREENWICH

Alliance with FPT Education

Section 2: Kubernetes Fundamentals

What is Kubernetes (K8s) ?

The Container Orchestrator

Definition: An open-source system for automating deployment, scaling, and management of containerized applications.

History: Developed by Google (Project Borg), donated to CNCF.

Analogy: If Docker is a single musician, Kubernetes is the Conductor of the orchestra.

Core Kubernetes Components

Pods: Smallest deployable unit (runs one or more containers).

Deployment: Manages a set of pods and ensures the desired state (e.g., 3 replicas).

Service: Exposes pods to the network and balances traffic between them.

Ingress: Manages external access to the services (e.g., routing based on URL).

Kubernetes in Production

- Kubernetes automates the **scaling** and **self-healing** of applications.
- Ensures **high availability** and **load balancing** across containers.
- Supports **rolling updates** and **blue/green deployments**.
- Commonly used in production environments like **Google Cloud**, **AWS,** and **Azure.**

Pet vs Cattle

Mindset Shift

Traditional (Pets): Servers have names (Zeus, Apollo). If they get sick, we nurse them back to health.

Cloud Native (Cattle): Servers are numbered (Node-01, Node-02). If they get sick, we replace them.

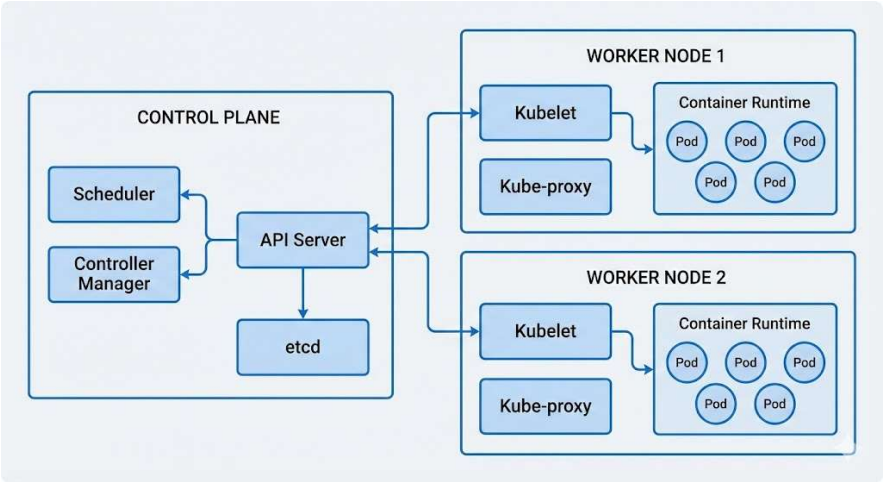
K8s treats containers like cattle.

K8S Architecture

How K8s works ?

Control Plane (Master): The Brain. Makes decisions (Scheduling, API).

Worker Nodes: The Muscle. Runs the actual applications.



Core Object: 1 - Pod

- The smallest deployable unit.
- A "wrapper" around a container.
- Usually 1 Container per Pod.
- Pods are ephemeral (mortal).
- They get IP addresses, but they change if the Pod restarts.

Core Object: 2 - Deployment

- Manages the Pods.
- **Desired State:** "*I want 3 replicas of Nginx*".
- **Self-Healing:** If a Pod crashes, the Deployment notices and starts a new one automatically to maintain the count of 3.

Scaling Power

Manual Scaling: kubectl scale deployment myapp --replicas=10.

Auto Scaling (HPA): "If CPU usage > 80%, add more pods".

This creates true Elasticity.

Core Object: 3 - Service

- Since Pod IPs change, how do we talk to them?
- **Service:** A stable network abstraction (Virtual IP).
- Acts as an internal Load Balancer.

Ingress (Routing)

How does the outside world reach the Service?

Ingress: Manages external access (HTTP/HTTPS).

Routes ***api.example.com*** to API-Service and example.com to Frontend-Service.

Declarative Configuration

YAML Manifests

In K8s, you don't give commands ("Run this").

You declare the desired state
("Here is a file describing what I want").

K8s works to make the Reality match the YAML.

Example Manifest

deployment.yaml

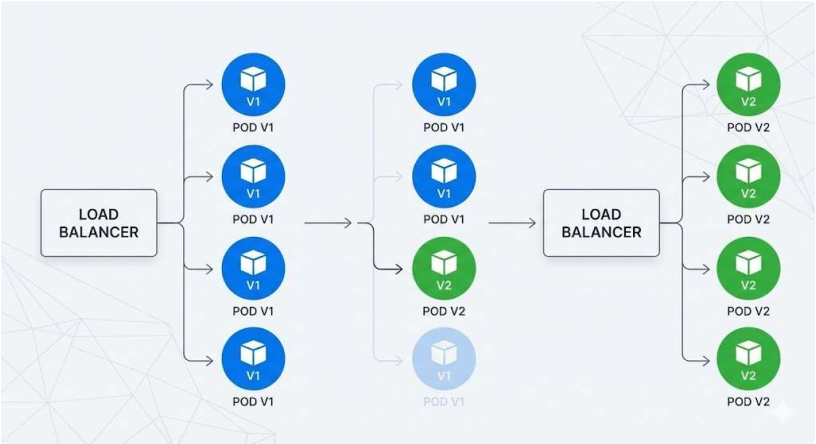
- This **Kubernetes Deployment YAML** defines:
  1. **apiVersion: apps/v1** – Uses the stable apps/v1 API for Deployments.
  2. **kind: Deployment** – Tells Kubernetes this object manages stateless Pods.
  3. **metadata: name: todo-app** – Names the Deployment todo-app.
  4. **spec: replicas: 3** – Runs 3 replicas (Pods) of the application.
  5. **selector: matchLabels: app: todo-app** – Selects Pods that have the label app: todo-app.
  6. **template: containers: todo-app** – Defines a container named todo-app using the image <your-dockerhub-username>/devops-todo:latest and exposing port 8080

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: todo-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: todo-app
  template:
    metadata:
      labels:
        app: todo-app
    spec:
      containers:
        - name: todo-app
          image: <your-dockerhub-username>/devops-todo:latest
          ports:
            - containerPort: 8080
```

Rolling Updates

Zero Downtime Deployment

- K8s replaces Pods incrementally.
- The Service only sends traffic to running Pods.
- Result: Users never see an error page during updates.



Helm

Package Manager

K8s YAML files can get complex.

Helm: The "NPM" or "APT" for Kubernetes.

Install complex apps (Prometheus, Database Clusters) with one command:
helm install my-db

Minikube

Learning K8s Locally

Running a full cluster is expensive.

Minikube: Runs a single-node cluster inside a VM/Docker on your laptop.

Perfect for learning and testing.

UNIVERSITY OF GREENWICH

Alliance with FPT Education

Section 3: Docker Compose vs Kubernetes

Docker Compose vs Kubernetes

Scope: Single Host (Compose) vs Multi-Host Cluster (K8s).

Configuration: Simple YAML (Compose) vs Verbose Manifests (K8s).

Features: Basic networking (Compose) vs Auto-scaling, Self-healing, Secrets, ConfigMaps (K8s).

Choosing the right tool

Use Compose: Local development, simple prototyping, small non-critical apps.

Use Kubernetes: Production microservices, high-traffic apps, enterprise systems needing 99.99% uptime.

The Complexity Cost

"Kubernetes is hard."

It requires significant maintenance overhead.

Don't use a bazooka to kill a mosquito.
For this course, Compose is enough for CD.

Summary of Concepts

- CD automates the release.
- K8s orchestrates containers.
- Pod = Container.
- Deployment = Manager.
- Service = Network/IP.

Section 4: Wrap Up

Lab 4 Preview

Automated CD

Goal: Build the final piece of the pipeline.

Steps:

SSH into provided Server.

Generate Keys.

Configure GitHub Secrets (HOST, KEY).

Write the deploy job in YAML.

Final Project Reminder

Next week is last lesson - Group Project Presentation.

Ensure your group has a working End-to-End Pipeline.

Presentation Tips

Don't just read slides.

Show, Don't Tell: Do a live demo of the pipeline running.

Show the "**Fail -> Fix -> Pass**" flow.

UNIVERSITY OF GREENWICH

Alliance with FPT Education